# Installationsanleitung für den IT-Support

Diese Anleitung richtet sich an die Systembetreuung. Lehrkräfte und Schüler:innen haben keine lokalen Administratorrechte.

**Was verteilt wird:** der Ordner `SpaceProject` aus dem Paket `SpaceProject.zip`. Er ist der Arbeitsordner der Schüler:innen und enthält die Bibliothek PyFoot bereits. Das Paket bekommen Sie von der Lehrkraft; die neueste öffentliche Fassung liegt unter https://github.com/SAE-GMO/pyfoot-space/releases (oberster Eintrag, Datei `SpaceProject.zip`).

**Pfadangaben:** `<SPACE-ORDNER>` steht für den Ordner, in den das Paket entpackt wurde, z. B. `C:\Schule\SpaceProject`. Relative Pfade gelten ab dort: `tools\check_environment.py` ist also `C:\Schule\SpaceProject\tools\check_environment.py`.

---

## 1. Kurzfassung

Zu installieren ist wenig:

| Was | Version | Für wen | Adminrechte |
|---|---|---|---|
| Python | 3.11 oder neuer | alle | ja, für systemweite Installation |
| Paket `pygame` | 2.5 oder neuer | alle | ja, für systemweite Installation |
| Paket `mypy` | aktuell | alle | ja, für systemweite Installation |
| VS Code | aktuell | alle | ja |
| Erweiterung `ms-python.python` | aktuell | alle | nein, je Benutzerprofil |
| Erweiterung `ms-python.vscode-pylance` | aktuell | alle | nein, je Benutzerprofil |
| Pakete `pytest`, `markdown`, `xhtml2pdf` | aktuell | nur Lehrkräfte | ja |

**Pylance ist nicht optional.** Diese Erweiterung erzeugt die Typwarnungen, die Schüler:innen beim Schreiben im Editor sehen. Sie wird normalerweise zusammen mit `ms-python.python` mitinstalliert, fehlt aber in abgewandelten VS-Code-Varianten. Ohne sie bleiben Typfehler unbemerkt — deshalb bitte ausdrücklich prüfen.

**mypy wird auf allen Arbeitsplätzen gebraucht.** Es liefert dieselben Hinweise wie Pylance, nur auf der Konsole — Schüler:innen rufen es über `python tools\check_code.py` auf. Auf Rechnern von Lehrkräften dient es zusätzlich der Qualitätssicherung des Projekts selbst.

**PyFoot wird NICHT installiert.** Die Lernbibliothek liegt als Ordner `pyfoot\` im Arbeitsordner `SpaceProject` und wird mit ihm verteilt. Das ist Absicht: Die Bibliothek wird im Laufe des Schuljahres weiterentwickelt, und jede Änderung soll ohne Zutun des Supports möglich sein — es genügt, den Ordner `SpaceProject` auszutauschen.

**Nicht nötig:** Der Ordner `SpaceProject` wird **nicht** installiert. Er läuft direkt, sobald Python, `pygame` und `mypy` vorhanden sind. Es sind keine Systemdienste, keine Registry-Einträge und keine Umgebungsvariablen außer dem PATH-Eintrag von Python erforderlich.

**Kein Internetzugang der Endgeräte nötig,** wenn nach Abschnitt 6 offline installiert wird.

---

## 1a. Der einfachste Weg: portables VS Code mit eigenem Python

**Wenn möglich, diesen Weg wählen — er kommt ganz ohne Administratorrechte aus.**

VS Code gibt es als ZIP-Archiv, das sich in einen beliebigen Ordner entpacken lässt und kein Installationsprogramm braucht. Eine solche Ausgabe kann ein eigenes Python mitbringen. Ist das der Fall, entfällt fast alles, was in den Abschnitten 3 bis 5 steht.

Prüfen, ob ein Python mitgeliefert ist:

```
dir <VSCODE-ORDNER>\python\python.exe
```

Falls vorhanden, die Pakete direkt dorthin installieren:

```
<VSCODE-ORDNER>\python\python.exe -m pip install pygame mypy pytest markdown xhtml2pdf
```

Das gelingt ohne Adminrechte, weil der Ordner dem Benutzer gehört. Danach in VS Code diese Python-Version auswählen:

*Strg* + *Umschalt* + *P*, dann *Python: Interpreter auswählen*, dann den Pfad `<VSCODE-ORDNER>\python\python.exe` wählen.

**Verteilung:** Der komplette VS-Code-Ordner lässt sich mitsamt Paketen kopieren. Ein Arbeitsplatz ist damit ohne jede Installation einsatzbereit.

> **Vorsicht bei zwei VS-Code-Installationen.** Ist zusätzlich eine normale Version vorhanden, verwenden beide getrennte Erweiterungen und getrennte Einstellungen. Die Prüfung aus Abschnitt 2 findet über den Suchpfad meist die installierte Version — nicht unbedingt die, mit

> der gearbeitet wird.

---

## 2. Vorher prüfen: Was fehlt überhaupt?

Vor jeder Installation feststellen, was bereits vorhanden ist. Das Skript ändert nichts, es meldet nur den Zustand und nennt anschließend genau die noch nötigen Befehle.

```
cd <SPACE-ORDNER>
```

```
python tools\check_environment.py
```

Ist Python noch gar nicht installiert, schlägt bereits der Aufruf fehl — dann mit Abschnitt 3 beginnen.

Das Skript endet mit Rückgabewert `0`, wenn alles Nötige vorhanden ist, sonst mit `1`. Es lässt sich damit auch in Verteilungsskripte einbinden.

---

## 3. Python installieren (Adminrechte)

### Empfohlen: über winget, systemweit

```
winget install --id Python.Python.3.13 --scope machine `
   --accept-package-agreements --accept-source-agreements
```

### Alternativ: Installationsdatei, unbeaufsichtigt

Herunterladen von https://www.python.org/downloads/windows/ (Datei `python-3.13.x-amd64.exe`), dann:

```
.\python-3.13.x-amd64.exe /quiet InstallAllUsers=1 PrependPath=1 `
   Include_pip=1 Include_test=0
```

Bedeutung der Schalter:

| Schalter | Wirkung |
|---|---|
| `InstallAllUsers=1` | systemweit nach `C:\Program Files\Python313` |
| `PrependPath=1` | trägt Python in den PATH ein — **wichtig**, sonst findet die Konsole `python` nicht |
| `Include_pip=1` | installiert die Paketverwaltung mit |

| | |
|---|---|
| `Include_test=0` | spart die Testsuite von Python selbst |

**PATH prüfen**

Nach der Installation eine **neue** Konsole öffnen und prüfen:

```
python --version
```

Erwartet wird `Python 3.11.x` oder neuer. Erscheint stattdessen der Microsoft-Store oder eine Fehlermeldung, fehlt der PATH-Eintrag. Dann manuell ergänzen (Adminrechte, systemweit):

```
setx /M PATH "%PATH%;C:\Program Files\Python313;C:\Program Files\Python313\Scripts"
```

Anschließend alle Konsolen und VS Code neu starten.

> **Hinweis zum Microsoft Store:** Windows 11 bringt einen Platzhalter mit, der beim Aufruf von `python` den Store öffnet. Er lässt sich abschalten unter *Einstellungen*, dann *Apps*, dann *Erweiterte App-Einstellungen*, dann *App-Ausführungsaliase*; dort `python.exe` und `python3.exe` deaktivieren.

---

## 4. Pakete installieren (Adminrechte)

Systemweit installieren, damit Lehrkräfte und Schüler:innen nichts weiter tun müssen. Konsole **als Administrator** öffnen.

Pflicht für alle Arbeitsplätze:

```
python -m pip install --upgrade pip
```

```
python -m pip install pygame mypy
```

Zusätzlich nur auf Rechnern von Lehrkräften:

```
python -m pip install pytest markdown xhtml2pdf
```

Prüfen:

```
python -c "import pygame; print(pygame.version.ver)"
```

---

## 5. VS Code (Adminrechte)

```
winget install --id Microsoft.VisualStudioCode --scope machine `
   --accept-package-agreements --accept-source-agreements
```

Beide benötigten Erweiterungen bereitstellen:

```
code --install-extension ms-python.python
```

```
code --install-extension ms-python.vscode-pylance
```

Der zweite Befehl ist meist überflüssig, weil Pylance mit der Python-Erweiterung mitkommt — schadet aber nicht und stellt sicher, dass die Typwarnungen im Editor tatsächlich erscheinen.

Prüfen:

```
code --list-extensions
```

In der Ausgabe müssen `ms-python.python` **und** `ms-python.vscode-pylance` stehen.

Wird VS Code systemweit installiert, müssen die Erweiterungen je Benutzerprofil vorhanden sein. Für eine zentrale Bereitstellung lässt sich der Ordner `%USERPROFILE%\.vscode\extensions` per Anmeldeskript oder Gruppenrichtlinie vorbelegen.

> **Hinweis:** Pylance darf lizenzrechtlich nur in offiziellen VS-Code-Builds von Microsoft verwendet werden. Kommt an der Schule VSCodium oder eine andere Abwandlung zum Einsatz, steht Pylance nicht zur Verfügung. Dann bitte Rückmeldung geben — in diesem Fall muss die Typprüfung im Unterricht anders gelöst werden.

---

## 6. Ohne Internetzugang der Endgeräte

Auf einem Rechner **mit** Internetzugang und gleicher Windows- und Python-Version die Pakete einmalig herunterladen:

```
python -m pip download pygame mypy pytest markdown xhtml2pdf -d D:\pyfoot_pakete
```

Den Ordner auf die Zielrechner kopieren, dort installieren:

```
python -m pip install --no-index --find-links D:\pyfoot_pakete pygame mypy
```

---

## 7. Falls keine Adminrechte verfügbar sind

Diese Wege funktionieren im Benutzerkontext, sind aber **nur Rückfallebene**. Für den Regelbetrieb ist die systemweite Installation nach Abschnitt 3 und 4 vorzuziehen, weil sonst jedes Benutzerprofil eigene Kopien anlegt.

### Python im Benutzerkontext

```
winget install --id Python.Python.3.13 --scope user `
   --accept-package-agreements --accept-source-agreements
```

### Pakete im Benutzerkontext

```
python -m pip install --user pygame mypy
```

Die Pakete landen unter `%APPDATA%\Python\Python313\site-packages`. Bei servergespeicherten Profilen erhöht das die Profilgröße — bitte einplanen.

### Virtuelle Umgebung im Projektordner

Sauberste Lösung ohne Adminrechte, aber je Kopie des Projekts einmal nötig:

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

```
python -m pip install pygame mypy
```

VS Code erkennt `.venv` im Projektordner selbstständig und bietet sie zur Auswahl an.

> **Häufige Stolperfalle:** VS Code legt manchmal ein leeres `.venv` an und wählt es aus. Dann fehlen dort **alle** Pakete, obwohl sie systemweit vorhanden sind — der Play-Knopf scheitert, das Terminal funktioniert. Entweder die Pakete in die Umgebung nachinstallieren oder das `.venv` löschen und eine andere Python-Version auswählen.

---

## 8. Abnahme

Auf einem eingerichteten Arbeitsplatz mit einem **normalen Benutzerkonto** prüfen:

```
cd <SPACE-ORDNER>
```

```
python tools\check_environment.py
```

Alle Pflichtpakete müssen `[ OK ]` melden. Danach der eigentliche Funktionstest:

```
python main_space.py
```

Es muss sich ein Fenster öffnen, in dem ein Raumschiff über ein Gitter fliegt. Schließen mit `Esc`. Damit ist der Arbeitsplatz einsatzbereit.

---

## 9. Häufige Störungen

| Symptom | Ursache | Abhilfe |
|---|---|---|
| `python` öffnet den Microsoft Store | App-Ausführungsalias aktiv | siehe Hinweis in Abschnitt 3 |

| | | |
|---|---|---|
| `'python' is not recognized` | PATH-Eintrag fehlt | Abschnitt 3, „PATH prüfen" |
| `ModuleNotFoundError: No module named 'pygame'` | Paket fehlt oder in anderem Python installiert | `python -m pip install pygame` mit **demselben** `python` wie im Fehler |
| `ModuleNotFoundError: No module named 'pyfoot'` | der mitgelieferte Ordner `pyfoot\` fehlt im Arbeitsordner | Ordner `SpaceProject` vollständig neu verteilen |
| `ModuleNotFoundError: No module named 'space'` | Konsole steht im falschen Ordner | `cd` in den Projektordner |
| `ModuleNotFoundError: No module named 'space'`, obwohl der Ordner da ist | **der Unterordner heißt `Space` statt `space`.** Beim Entpacken oder Verschieben schreibt Windows Ordnernamen mitunter groß. Der Dateidienst ist zwar groß-klein-blind, Python prüft die Schreibung aber genau | Unterordner in Kleinbuchstaben umbenennen — siehe Hinweis unter der Tabelle |
| Play-Knopf scheitert, Terminal funktioniert | VS Code nutzt eine andere Python-Version, oft ein leeres `.venv` | *Strg* + *Umschalt* + *P*, dann *Python: Interpreter auswählen* |
| PowerShell fragt „Führen Sie ausschließlich vertrauenswürdige Skripts aus" | Zonenmarkierung heruntergeladener Dateien | ohne Adminrechte: `Unblock-File -Path "<Pfad aus der Warnung>"` |
| mypy meldet `Duplicate module named "__main__"` | Konsole steht im falschen Ordner; mypy meldet das irrefuehrend | `cd` in den Projektordner, oder `python tools\check_project.py` verwenden — das funktioniert von ueberall |
| Mehrere Python-Versionen parallel | verschiedene Installationen | zeigt die genutzte Installation: `python -c "import sys; print(sys.executable)"` |

| | | |
|---|---|---|
| Fenster öffnet sich nicht, Fehler zu SDL | Grafiktreiber oder Sitzung ohne Bildschirm | auf einem Arbeitsplatz mit Bildschirm testen |
| Doppelklick auf eine Klasse öffnet nichts | kein Editor gefunden — meist ein aus dem ZIP entpacktes VS Code, das nicht im Suchpfad steht | `PYFOOT_EDITOR` setzen, siehe unten |

**Wenn der Doppelklick keinen Editor öffnet**

Ein Doppelklick auf eine Klasse soll ihre Datei öffnen — und zwar in dem VS Code, mit dem gerade gearbeitet wird. PyFoot ruft dafür `code -r -g datei:zeile` auf; `-r` öffnet im **schon offenen Fenster**, statt ein zweites aufzumachen. Läuft noch kein VS Code, startet es eines.

Gesucht wird der Reihe nach:

1. den Eintrag in `PYFOOT_EDITOR`
2. das VS Code, aus dessen Terminal gestartet wurde — die laufende Sitzung
3. `code` im Suchpfad
4. die üblichen Orte einer VS-Code-Installation
5. die Standardanwendung des Systems für `.py`

Punkt 2 steht vorn, damit bei **zwei** vorhandenen Versionen (Abschnitt 1a warnt davor) die benutzte gewinnt. Erkannt wird sie an einer Umgebungsvariablen, die VS Code in seinem Terminal setzt — wer PyFoot aus dem Explorer startet, landet bei Punkt 3 oder 4.

**Ein aus dem ZIP entpacktes VS Code (Abschnitt 1a) steht in keinem Suchpfad.** Wird es nicht gefunden, trägt man es einmal ein:

```
setx PYFOOT_EDITOR "<VSCODE-ORDNER>\bin\code.cmd"
```

`setx` schreibt die Angabe dauerhaft in das Benutzerprofil; **neue** Konsolen kennen sie danach. Für die laufende Sitzung zusätzlich:

```
$env:PYFOOT_EDITOR = "<VSCODE-ORDNER>\bin\code.cmd"
```

Prüfen, ob PyFoot etwas findet — der Befehl nennt den Fund oder `None`:

```
python -c "from pyfoot.editor import source; print(source.find_editor())"
```

> **VS Code muss dafür nicht installiert werden.** `PYFOOT_EDITOR` nimmt jedes Programm an — Notepad++, Thonny, IDLE. Nur der Sprung zur Zeile bleibt VS Code vorbehalten; andere Editoren öffnen die Datei am Anfang. Ist gar nichts eingetragen, öffnet Windows die Datei mit der Anwendung, die für `.py` hinterlegt ist. Die Statuszeile nennt in jedem Fall Datei und Zeile.

### Wenn ein Ordner plötzlich großgeschrieben ist

Der Paketordner `space` ist kleingeschrieben. Beim Entpacken und Verschieben kann Windows daraus `Space` machen — vor allem, wenn ein Ordner gleichen Namens darüber liegt. Deshalb heißt der Projektordner seit November 2026 `SpaceProject`; ältere Pakete hießen noch `Space`. Sichtbar ändert sich fast nichts, und der Dateidienst stört sich nicht daran; **Python schon.** Die Meldung nennt dann nur das fehlende Modul, nicht den Grund.

So sieht man die tatsächliche Schreibung:

```
Get-ChildItem -Directory | Select-Object Name
```

Umbenennen geht **nicht in einem Schritt**: Der direkte Versuch scheitert mit „Der Quell- und der Zielpfad dürfen nicht identisch sein", weil Windows `Space` und `space` für denselben Namen hält. Es braucht einen Zwischenschritt:

```
Rename-Item Space -NewName zwischenschritt; Rename-Item zwischenschritt -NewName space
```

Danach zeigt `Get-ChildItem -Directory` den Namen klein. Denselben Weg gibt es für `ships`, `levels` und `pyfoot` — alle vier sind kleingeschrieben, nur der Projektordner darüber heißt `SpaceProject`.